

Baseline Chart & Email Report

Complete Technical Walkthrough

The Baseline Chart & Email Report — Complete Technical Walkthrough

Purpose. You will be explaining these two features to your lead software engineer. This document assumes zero prior knowledge of Polymer or the JavaScript patterns used, and builds up from first principles. Every line of code that matters is quoted and explained. Two worked examples trace real data all the way through. A glossary and an anticipated-questions section are included at the end for the conversation itself.

PART 0 — The five ideas you need before anything else

1. Polymer components are custom HTML tags. `<fid-analytical-dashboard>` is one. Inside it: a `<template>` (what's on screen), a properties object (the component's variables), and methods (its functions).

2. Data binding removes manual DOM updates. Write a property name inside `[[ ]]` in the HTML, and Polymer keeps that spot in sync automatically. You never write `document.getElementById(...).innerText = ....`

3. Two binding directions.

- `[[foo]]` — **one-way**, parent → child. "Here is a value, display it."
- `{{foo}}` — **two-way**. The child can write a value *back* into the parent's property. Used whenever an element (a query, or a form input) needs to push data *up* into the component.

4. A computed property is a value Polymer derives for you. You declare its dependencies; whenever any of them change, Polymer automatically re-runs a function and stores the result. You never call it by hand.

5. `this` in JavaScript changes meaning inside callbacks. Every function(){...} passed to `.then()`, `.map()`, `.forEach()`, or `setTimeout` gets its **own** `this`, which is *not* the component. The universal fix seen throughout this code: `var self = this;` before the callback, then use `self` inside it.

Keep these five in your head — every explanation below leans on them.

PART A — THE BASELINE CHART

(Slot 6)

A1. The data queries — where the numbers come from

```
<!-- ===== SLOT 6: BASELINE (fully independent range) =====
      These two feeds are deliberately bound to
baselineStart/baselineEnd,
      NOT to the page's period selector. -->
<nuxeo-repository-data
  start-date="[[baselineStart]]"
  end-date="[[baselineEnd]]"
  with-date-intervals="month"
  date-field="dc:created"
  data="{{baselineDocsPerMonth}}"
  index="[[index]]">
</nuxeo-repository-data>

<nuxeo-repository-data
  start-date="[[baselineStart]]"
  end-date="[[baselineEnd]]"
  with-date-intervals="month"
  date-field="dc:created"
  metrics="sum(file:content.length)"
  data="{{baselineStoragePerMonth}}"
  index="[[index]]">
</nuxeo-repository-data>
```

`<nuxeo-repository-data>` is **invisible** — it draws nothing. It is a Nuxeo-provided element whose only job is: "ask Elasticsearch a question, put the answer in a property."

Line by line:

- `start-date="[[baselineStart]]" / end-date="[[baselineEnd]]"` — these two properties are the *entire* reason the baseline is independent from the page. The page's Overall/By-Type/By-Format charts bind to `startDate / endDate` (controlled by the period dropdown). The baseline feeds bind to their **own** pair, `baselineStart / baselineEnd`, which are changed *only* by the baseline's own Apply button. Change the page period → these two queries are completely unaffected. That is the decoupling your lead will ask about.
- `with-date-intervals="month"` — tells Elasticsearch to **bucket** the results by month instead of returning one grand total. This is what turns a single number into a month-by-month series.
- `date-field="dc:created"` — which date field to bucket on (`dc:created` = Dublin Core "date created").
- `metrics="sum(file:content.length)"` (*second query only*) — without this, the query counts **documents**. With it, the query instead **sums** the byte-size of every file. Two nearly-identical queries, one counting, one summing — that's why there are two elements, not one.
- `data="{{baselineDocsPerMonth}}"` — where the answer lands. This **must** be `{{ }}` (two-way), because the query result needs to flow *upward* into the component's property. Every other binding you'll see for *display* uses `[[ ]]`; this is the one direction that needs `{{ }}`.
- `index="[[index]]"` — which Elasticsearch index to query, passed through from the component's own configuration.

What comes back, for each query, is an array of month buckets:

```
baselineDocsPerMonth = [
  { key: '2026-07', value: 1000 },
  { key: '2026-08', value: 1500 }
```

```

]
baselineStoragePerMonth = [
  { key: '2026-07', value: 524288000 },
  { key: '2026-08', value: 786432000 }
]

```

Every entry has a **key** (the month) and a **value** (the number). Remember this shape — it is the input to every helper below.

A2. The properties — the component's memory

```

baselineStart:      { type: String, value: '2026-07-01' },
baselineEnd:        { type: String, value: '2026-07-31' },
baselinePick:       { type: String, value: '2026-07' },
baselineEndPick:    { type: String, value: '2026-07' },
baselineLabel:      { type: String, value: 'July 2026' },
baselineEndLabel:   { type: String, value: 'July 2026' },
baselinePickStatus: { type: String, value: '' },
baselineDocsPerMonth: { type: Array, value: function() { return
[]; } },
baselineStoragePerMonth: { type: Array, value: function() { return
[]; } },

_baselineGrowth: {
  type: Array,
  value: function() { return []; },
  computed: '_computeBaselineGrowth(baselineStoragePerMonth,
baselineDocsPerMonth)'
},

```

Six string/array properties, in two families:

Property	Role
baselineStart / baselineEnd	drive the query (YYYY-MM-DD) — only changed by Apply
baselinePick / baselineEndPick	the raw <input type="month"> values (YYYY-MM) — change on every keystroke
baselineLabel / baselineEndLabel	human-readable text for headings ('July 2026')
baselinePickStatus	validation/info message shown next to Apply
baselineDocsPerMonth / baselineStoragePerMonth	raw query results

Why two separate pairs for start (baselineStart vs baselinePick)?

This is the scale-safety mechanism. The <input> is bound to baselinePick, which changes on **every keystroke**. If the query were bound directly to that, every keystroke would re-fire an Elasticsearch query. Instead, the query is bound to baselineStart, which is touched in exactly one place — inside _applyBaseline() — so typing in the box costs nothing, and a query only fires when the user explicitly commits.

value: function() { return []; } — not value: []. This is a **mandatory Polymer rule** for any property whose default is an object or array. If you wrote value: [] directly, every instance of the component would share the *same* array in memory — a classic bug. Wrapping it in a function makes Polymer call it fresh for each instance, so each gets its own independent array.

The computed property, _baselineGrowth:

```

computed: '_computeBaselineGrowth(baselineStoragePerMonth,
baselineDocsPerMonth)'

```

This line means: *"_baselineGrowth is not something I set directly. Watch baselineStoragePerMonth and baselineDocsPerMonth; whenever either changes, automatically call _computeBaselineGrowth() with both as arguments, and store whatever it returns."* This is **reactive programming** — nobody ever calls `_computeBaselineGrowth()` by hand.

A3. The `ready()` lifecycle method

```
ready: function() {
  this._computeDates(this.selectedPeriod);
  // Default the baseline range end to the current month so the
first
  // render covers baseline -> today. The user can change it and
Apply.
  var nowMonth = this._toISODate(new Date()).substring(0, 7);
  this.setProperties({
    baselineEndPick: nowMonth,
    baselineEnd: this._monthEndDate(nowMonth),
    baselineEndLabel: this._monthYearLabel(nowMonth)
  });
},
```

`ready()` is a **Polymer lifecycle callback** — it runs automatically once, the moment the component is fully initialised and attached to the page. It's the natural place to set up initial state.

- `new Date()` — today's date, from the browser.
- `.substring(0, 7)` — an ISO date string like '2026-07-19' sliced down to '2026-07' (just year-month). A plain string operation, not a date library call.
- `this.setProperties({...})` — sets **three properties in a single notification cycle**. This is a real Polymer performance feature: if you wrote `this.baselineEndPick = x; this.baselineEnd = y;` as two separate statements, Polymer would fire two separate change-notification passes — and if any query is bound to both, it would **fire twice**. `setProperties` batches them into one pass. This exact pattern is used everywhere dates change in this component, specifically to prevent doubled Elasticsearch queries. This was a real, found-and-fixed performance bug — worth mentioning to your lead.

A4. The card markup

```
<nuxeo-card class="full-width" heading="Growth vs Baseline">
  <div class="chart-sub-title">
    [[_baselineSubtitle(baselineLabel, baselineEndLabel)]]
  </div>

  <div class="baseline-picker">
    <div class="date-field">
      <label>Baseline month</label>
      <input type="month" value="{{baselinePick::change}}" />
    </div>
    <div class="date-field">
      <label>Up to</label>
      <input type="month" value="{{baselineEndPick::change}}" />
    </div>
    <div class="date-field">
      <label>&nbsp;</label>
      <button type="button" class="custom-submit" on-
tap="_applyBaseline">Apply</button>
    </div>
    <span class="baseline-picker-status">[[baselinePickStatus]]
  </span>
```

```

</div>

<div class="chart-box">
  <chart-bar
    labels="[[_monthLabels(baselineStoragePerMonth)]]"
    values="[[_baselineValues(baselineStoragePerMonth,
baselineDocsPerMonth)]]"
    series="[[_baselineSeries(baselineStoragePerMonth)]]"
    options="[[_overallOptions(baselineStoragePerMonth)]]">
  </chart-bar>
</div>

<template is="dom-if" if="[[_hasBaseline(_baselineGrowth)]]">
  <div class="baseline-table-wrap">
    <div class="chart-sub-title">Change vs [[baselineLabel]]</div>
    <table class="baseline-table">
      <thead>
        <tr>
          <th>Month</th><th>Documents</th>
          <th>vs [[_baselineShort(baselineLabel)]]</th>
          <th>Storage</th>
          <th>vs [[_baselineShort(baselineLabel)]]</th>
        </tr>
      </thead>
      <tbody>
        <template is="dom-repeat" items="[[_baselineGrowth]]">
          <tr>
            <td>[[item.month]]</td>
            <td class="num-cell">[[item.docs]]</td>
            <td class$="[[_deltaClass(item.docsPct)]]">
[[item.docsPct]]</td>
            <td class="num-cell">[[item.storage]]</td>
            <td class$="[[_deltaClass(item.storagePct)]]">
[[item.storagePct]]</td>
          </tr>
        </template>
      </tbody>
    </table>
  </div>
</template>
</nuxeo-card>

```

`<input type="month" value="{baselinePick::change}" />` — a native HTML5 month picker (browser-rendered, no JS needed for the calendar UI). The `::change` suffix is Polymer's syntax for **event-based two-way binding**: "whenever this input fires a change event, write its value back into baselinePick." Two of these — one for the start month, one ("Up to") for the end month — giving the baseline card two independent date pickers, fully separate from the page's own date range widget above.

`on-tap="_applyBaseline"` — Polymer's tap/click handler binding. Calls the method when the button is pressed.

`<template is="dom-if" if="[[_hasBaseline(_baselineGrowth)]]">` — conditional rendering. If the expression is false, the block is **not created in the DOM at all** — not hidden with CSS, genuinely absent. This keeps an empty table off the screen before the first query returns.

`<template is="dom-repeat" items="[[_baselineGrowth]]">` — a loop. Stamps one copy of its contents for every element in the `_baselineGrowth` array. Inside, `item` is a magic variable referring to the current element, so `[[item.month]]` reads that row's month field. Five array entries → five `<tr>` rows on screen, automatically, with no manual DOM manipulation.

`class$="[[_deltaClass(item.docsPct)]]"` — **note the \$**. This is the one Polymer beginner trap worth flagging explicitly: `class="..."` sets a JavaScript *property* on the element; `class$="..."` sets the actual HTML *attribute*. For `class`, `style`, `href`, and similar native HTML attributes, you **must** use `$=`, or the CSS class silently never appears and nothing turns green/red. This is exactly what colours the percentage cells.

A5. `_applyBaseline()` — the Apply button's logic

```
_applyBaseline: function() {
  var pick    = (this.baselinePick || '').trim();    // 'YYYY-MM'
  var endPick = (this.baselineEndPick || '').trim(); // 'YYYY-MM'
  var MONTH = /^d{4}-d{2}$/;

  if (!MONTH.test(pick)) {
    this.baselinePickStatus = 'Pick a baseline month first.';
    return;
  }
  if (!MONTH.test(endPick)) {
    this.baselinePickStatus = 'Pick an end month first.';
    return;
  }
  // String compare works on 'YYYY-MM' (zero-padded, fixed width).
  if (endPick < pick) {
    this.baselinePickStatus = 'End month cannot be before the
baseline month.';
    return;
  }

  this.setProperties({
    baselineStart: pick + '-01',
    baselineEnd:   this._monthEndDate(endPick),
    baselineLabel: this._monthYearLabel(pick),
    baselineEndLabel: this._monthYearLabel(endPick)
  });
  this.baselinePickStatus = '';
},
```

`var MONTH = /^d{4}-d{2}$/;` — a **regular expression**. Read as: start of string (^), exactly 4 digits, a literal hyphen, exactly 2 digits, end of string (\$). `MONTH.test(pick)` returns true/false for whether pick matches that shape. This guards against an empty or malformed picker value reaching the query.

`if (endPick < pick)` — comparing two strings with `<`. This works correctly here specifically because 'YYYY-MM' is a **fixed-width, zero-padded** format: JavaScript compares strings character by character, and because every field is the same width, string order and chronological order are identical ('2026-03' < '2026-09' is true, correctly). This trick breaks if the format isn't zero-padded — worth knowing why it's safe *here*.

The `setProperties` call is the scale-safety keystone of this whole feature. It commits `baselineStart` and `baselineEnd` together. Both of the baseline's Elasticsearch queries are bound to these two properties, so if they were set one at a time, each query would fire twice per click (once for start, once for end) — an unnecessary doubling of load, and worse, a transient query with a *mismatched* half-updated range. One `setProperties` call means: one query fire, always with a self-consistent date pair.

No upper or lower bound is enforced on the picked month. Any month, past or future, is accepted — the format check is the *only* gate. This was a deliberate choice: leadership can select any range they

want. If the range ends up empty (e.g. a future baseline with no documents yet), the chart simply renders empty rather than being blocked — nothing crashes, because every helper below has a guard clause for empty input (see A9–A10).

A6. `_monthEndDate()` — finding the last day of a month

```
_monthEndDate: function(pick) {
  if (!/^d{4}-d{2}$/.test(pick || '')) return '';
  var parts = pick.split('-');
  var last = new Date(parseInt(parts[0], 10), parseInt(parts[1],
10), 0);
  return this._toISODate(last);
},
```

The clever line: `new Date(year, month, 0)`. JavaScript's `Date` constructor treats **day 0** of a given month as *the last day of the previous month*. So passing the target month **unadjusted** (JS months are zero-indexed, so `parts[1]` — the 1-indexed month from the string — is already "next month, zero-indexed") lands exactly on the last real day of the picked month. This one line correctly handles 28-day February, 29-day leap-year February, and 30- vs 31-day months, with no lookup table and no special-casing.

A7. `_monthYearLabel()` — turning '2026-07' into 'July 2026'

```
_monthYearLabel: function(pick) {
  if (!/^d{4}-d{2}$/.test(pick || '')) return '';
  var names = ['January', 'February', 'March', 'April', 'May', 'June',
'July', 'August', 'September', 'October', 'November', 'December'];
  var parts = pick.split('-');
  var mi = parseInt(parts[1], 10) - 1;
  if (mi < 0 || mi > 11) return '';
  return names[mi] + ' ' + parts[0];
},
```

A guard clause, split the string on `-`, look up the month name by index (`- 1` because array indices are 0-based but months in the string are 1-based), concatenate with the year. Straightforward string plumbing — included here because it's what turns the raw picker value into the human-readable headings you see everywhere ("Growth vs January 2026").

A8. `_baselineSubtitle()` and `_baselineShort()`

```
_baselineSubtitle: function(label, endLabel) {
  var from = label || 'selected';
  if (endLabel && endLabel !== from) {
    return 'Documents and storage added per month, ' + from + ' to '
+ endLabel
    + ' (baseline: ' + from + ')';
  }
  return 'Documents and storage added per month, starting from the '
+ from + ' baseline';
},

_baselineShort: function(label) {
  if (!label) return 'base';
}
```

```
    return label.substring(0, 3);    // 'July 2026' -> 'Jul'
  },
```

`_baselineSubtitle` builds the descriptive line under the card heading — it branches on whether the range is a single month or a real span, so the text reads naturally either way. `_baselineShort` just takes the first three characters of the label for the compact table-header text ("vs Jul").

A9. The chart's data helpers

```

_baselineValues: function(storage, docs) {
  if (!storage || !storage.length) return [[]];
  var unit = this._storageUnit(storage);
  var storageVals = storage.map(function(entry) {
    var v = (entry.value || 0) / unit.divisor;
    return Math.round(v * 100) / 100;
  });
  var docMap = {};
  (docs || []).forEach(function(d) { docMap[d.key] = d.value || 0;
});
  var docVals = storage.map(function(entry) { return
docMap[entry.key] || 0; });
  return [storageVals, docVals];
},

_baselineSeries: function(storage) {
  return ['Storage added in (' + this._storageUnit(storage).name +
'),
        'Number of documents created'];
},
```

`if (!storage || !storage.length) return [[]];` — a guard clause. When the page first renders, the query hasn't returned yet, so `storage` is undefined or empty. Without this line, everything below would crash on undefined. Every helper that touches query results has one of these — Polymer calls binding helpers eagerly, including before data exists.

`var unit = this._storageUnit(storage);` — auto-scales the unit (KB/MB/GB/TB) based on the *actual* size of the data (see A12). Nothing is hardcoded to a fixed unit like "always MB" — the divisor and the label text both come from this one call, so they can never disagree with each other.

`storage.map(function(entry) {...})` — `.map()` is a built-in JavaScript array method: run a function on every element, collect the results into a **new** array (the original is untouched). Here: convert every month's raw byte count into the chosen unit, rounded to 2 decimals.

The alignment problem — the single subtlest piece of this whole feature. Storage and document counts come from **two separate queries**. They are not guaranteed to return the same set of months — if a month has zero documents, Elasticsearch may omit that bucket entirely from one query but not the other. If you naively zipped the two arrays by position (`docs[i]` paired with `storage[i]`), a missing month in one array would silently shift every later value out of alignment — August's document count could end up drawn against September's storage bar. Silently wrong data, with no error.

The fix, in three lines:

```

var docMap = {};
(docs || []).forEach(function(d) { docMap[d.key] = d.value || 0; });
var docVals = storage.map(function(entry) { return docMap[entry.key]
|| 0; });
```

1. Build a lookup object keyed by month: { '2026-07': 1000, '2026-

- 08': 1500 }.
2. Walk the **storage** array (chosen as the reference order) and, for each of *its* months, look up the matching document count by key.
 3. If a month is missing from the doc data, `|| 0` supplies a zero rather than crashing or misaligning.

Result: `docVals` is guaranteed the same length and order as `storageVals`, matched by **month**, never by array position.

return [storageVals, docVals]; — an array containing two arrays. Chart.js interprets this as two data series — the two grouped bars per month.

`_baselineSeries` returns the matching legend text, in the same order (index 0 labels the storage series, index 1 the document series). It reuses `_storageUnit` so the legend text and the actual divisor used in `_baselineValues` can never disagree.

Two more helpers are **shared** with the page's Overall chart (deliberately — same visual language, same axis behaviour):

```

    _monthLabels: function(data) {
      if (!data || !data.length) return [];
      var self = this;
      return data.map(function(entry) { return
self._toMonthYear(entry.key); });
    },

```

Turns every bucket's key into an x-axis label via `_toMonthYear` (A12).

`_overallOptions(storage)` returns the Chart.js configuration object (axis styling, legend, tooltip behaviour) — reused so the baseline chart looks *exactly* like the Overall chart, just with different data.

A10. `_computeBaselineGrowth()` — the percentage table's engine

```

_computeBaselineGrowth: function(storage, docs) {
  if (!storage || !storage.length) return [];
  var self = this;
  var docMap = {};
  (docs || []).forEach(function(d) { docMap[d.key] = d.value || 0;
});

  var baseDocs = docMap[storage[0].key] || 0;
  var baseStorage = storage[0].value || 0;

  function pct(current, base, fmt) {
    if (!base) {
      return current > 0 ? '+' + fmt(current) + ' new' : '0%';
    }
    var change = ((current - base) / base) * 100;
    var rounded = Math.round(change * 10) / 10;
    return (rounded > 0 ? '+' : '') + rounded + '%';
  }

  var fmtDocs = function(v) { return self._withCommas(v); };
  var fmtStorage = function(v) { return self._formatStorage(v); };

  return storage.map(function(entry, i) {
    var d = docMap[entry.key] || 0;
    var st = entry.value || 0;
    return {
      month: self._toMonthYear(entry.key),
      docs: self._withCommas(d),
      storage: self._formatStorage(st),
      docsPct: i === 0 ? 'baseline' : pct(d, baseDocs, fmtDocs),

```

```

        storagePct: i === 0 ? 'baseline' : pct(st, baseStorage,
fmtStorage)
    });
  });
},

```

var self = this; — the pattern from Part 0, point 5. Inside the nested function(entry, i) passed to .map(), this no longer refers to the component. self is the escape hatch that lets the callback still call self._toMonthYear(...) etc.

var baseDocs = docMap[storage[0].key] || 0; — this line **defines what "the baseline" means numerically**. storage[0] is whatever bucket the query returned *first*. Because the query's start-date is baselineStart, the first bucket returned **is** the baseline month, by construction — the code never needs to search for it or hardcode a month name. Change the picker, re-Apply, and the first bucket becomes whatever new month was chosen. No other line needs to change.

The percentage formula, and the zero-baseline fix:

```

if (!base) {
  return current > 0 ? '+' + fmt(current) + ' new' : '0%';
}

```

Standard percentage-change is (current - base) / base × 100. That is **undefined** when base is 0 — division by zero. Rather than showing a meaningless n/a, or silently producing NaN/Infinity, this branch shows the **absolute increase** instead: +116 new for documents, +452.9 KB new for storage. fmt is passed in so documents get comma-formatting and storage gets the right unit — the same pct() function correctly serves both columns.

If the baseline was zero **and** the current month is *also* zero, it returns '0%' — genuinely nothing changed, and 0% is the honest, correct answer there.

```

var change = ((current - base) / base) * 100;
var rounded = Math.round(change * 10) / 10;
return (rounded > 0 ? '+' : '') + rounded + '%';

```

The normal case: standard percentage-change formula, rounded to one decimal. The ternary prepends a + for growth (JavaScript's own - sign already appears for negative numbers, so no equivalent branch is needed there).

The .map() that builds the table rows returns, for every month, an object shaped like a table row: { month, docs, storage, docsPct, storagePct }. The second argument to the map callback, i, is the array **index** — used to special-case row 0 (i === 0 ? 'baseline' : ...), since comparing the baseline month to itself would always be a meaningless "+0%".

A11. _hasBaseline() and _deltaClass()

```

_hasBaseline: function(rows) {
  return !(rows && rows.length);
},

_deltaClass: function(pctText) {
  if (!pctText || pctText === 'baseline' || pctText === 'n/a')
return 'delta-cell delta-flat';
  if (pctText.charAt(0) === '+') return 'delta-cell delta-up';
  if (pctText.charAt(0) === '-') return 'delta-cell delta-down';
  return 'delta-cell delta-flat';
},

```

`_hasBaseline` — `!!` is a double-negation idiom that forces any value into a true boolean. Used by the `dom-if` to decide whether to render the table at all.

`_deltaClass` returns a **CSS class name**, chosen purely by inspecting the **first character** of the percentage text: `+` → green (delta-up), `-` → red (delta-down), anything else (including the new `'+116 new'` — which still starts with `+`, so it correctly goes green) → falls through correctly. The component decides *meaning*; the stylesheet decides *colour*.

A12. The shared formatting helpers

```
    _toMonthYear: function(key) {
      // handles epoch-ms keys, 'YYYY-MM', 'YYYY-MM-DD', and ISO
      datetimes,
      // normalising all of them to 'Jul-26' style labels
      ...
    },
    _withCommas: function(n) {
      n = n || 0;
      return String(n).replace(/\B(?=(\d{3})+(?!\d))/g, ',');
    },
    _formatStorage: function(bytes) {
      bytes = bytes || 0;
      if (bytes >= 1125899906842624) return (Math.round(bytes /
1125899906842624 * 10) / 10) + ' PB';
      if (bytes >= 1099511627776) return (Math.round(bytes /
1099511627776 * 10) / 10) + ' TB';
      if (bytes >= 1073741824) return (Math.round(bytes /
1073741824 * 10) / 10) + ' GB';
      if (bytes >= 1048576) return (Math.round(bytes / 1048576
* 10) / 10) + ' MB';
      if (bytes >= 1024) return (Math.round(bytes / 1024 *
10) / 10) + ' KB';
      return bytes + ' B';
    },
    _storageUnit: function(data) {
      var max = this._storageMax(data);
      if (max >= 1099511627776) return { name: 'TB', divisor:
1099511627776 };
      if (max >= 1073741824) return { name: 'GB', divisor: 1073741824
};
      if (max >= 1048576) return { name: 'MB', divisor: 1048576 };
      return { name: 'KB', divisor: 1024 };
    },
    _storageMax: function(data) {
      if (!data || !data.length) return 0;
      return data.reduce(function(max, entry) {
        var v = entry.value || 0;
        return v > max ? v : max;
      }, 0);
    },
    _toISODate: function(date) {
      return date.toISOString().split('T')[0];
    }
  }
```

`_withCommas` — a **regular expression** you don't need to memorise, but should be able to describe: it finds every position in the digit string that is followed by a group of three digits (and *not* at the very end), and inserts a comma there. `1234567` → `1,234,567`.

`_formatStorage` — a ladder of if checks from largest unit to smallest, each threshold being the exact byte count of that unit (1024^4 for TB, 1024^3 for GB, etc.). The **first** matching threshold wins, so a value is always shown in the *largest* unit that keeps the number ≥ 1 .

`_storageUnit / _storageMax` — this pair is what makes chart axes auto-scale. `_storageMax` uses `.reduce()` — a JavaScript array method that walks the array carrying an accumulator (max, starting at 0) and returns a single final value; here, the largest value seen across all buckets. `_storageUnit` then picks **one unit for the whole chart** based on that single largest value, so every bar in a given render uses the same divisor and the same legend label — consistent, never mixed units within one chart.

`_toISOString` — converts a JS Date object to the plain 'YYYY-MM-DD' string Nuxeo's REST API expects, by taking the ISO string and cutting off everything from the T (time) onward.

A13. The CSS

```
.baseline-picker {
  display: flex;
  align-items: flex-end;
  gap: 12px;
  margin: 6px 0 16px;
  flex-wrap: wrap;
}
.baseline-picker .date-field {
  display: flex;
  flex-direction: column;
  gap: 4px;
}
.baseline-picker input[type="month"] {
  height: 32px;
  padding: 0 10px;
  border: 1px solid #cfd8dc;
  border-radius: 6px;
  ...
}
.baseline-picker input[type="month"]:focus {
  outline: none;
  border-color: #43a047;
  box-shadow: 0 0 0 3px rgba(67, 160, 71, 0.15);
}

.baseline-table {
  width: 100%;
  border-collapse: collapse; /* single shared borders, not doubled
*/
  ...
}
.baseline-table th:nth-child(3),
.baseline-table td:nth-child(3) {
  border-right: 2px solid #20302a; /* divider: documents | storage
*/
}
.baseline-table .num-cell {
  font-variant-numeric: tabular-nums; /* every digit the same
width */
}
.delta-up { color: #2e7d32; } /* green – growth */
.delta-down { color: #c0392b; } /* red – decline */
.delta-flat { color: #9aa39c; } /* grey – baseline / no change */
```

Two choices your lead may probe:

`th:nth-child(3) / td:nth-child(3)` — the table's columns are Month(1), Documents(2), vs-baseline(3), Storage(4), vs-baseline(5). Targeting the **3rd** cell puts a vertical rule on the right edge of the documents block, visually separating "documents growth" from "storage growth" — pure CSS, no extra markup needed.

font-variant-numeric: tabular-nums — in most fonts, digits have slightly different widths (a 1 is narrower than an 8), so a column of numbers looks ragged. This forces every digit glyph to the same width, so figures line up vertically like a real financial table.

A14. The complete program flow

1. Page loads → ready() fires once
baselineStart = '2026-07-01' (default) baselineEnd = <this month's end> (computed)
|
2. Both <nuxeo-repository-data> elements fire their ES queries in parallel
Feed A: document COUNT per month, baselineStart → baselineEnd
Feed B: byte SUM per month, baselineStart → baselineEnd
|
3. Results land in properties (via {{ }} two-way binding)
baselineDocsPerMonth = [{key,value}, ...]
baselineStoragePerMonth = [{key,value}, ...]
|
4. Polymer reacts on two independent fronts:
|
| — CHART PATH
| _monthLabels() → x-axis labels
| _baselineValues() → [[storage...], [docs...]], unit-
scaled & key-aligned
| _baselineSeries() → legend text, matching unit
| _overallOptions() → axis/legend/tooltip config
| ↓
| <chart-bar> draws two grouped bars per month
|
| — TABLE PATH
| _baselineGrowth is `computed` → _computeBaselineGrowth()
auto-reruns
| ↓
| [{month, docs, storage, docsPct, storagePct}, ...]
| ↓
| dom-if → array non-empty? → render
| dom-repeat → one <tr> per row
| _deltaClass → colour each % cell
|
5. User picks a new baseline month / end month, clicks Apply
_applyBaseline() validates → setProperties({baselineStart, baselineEnd, ...})
|
| (one notification cycle → both ES queries fire exactly once)
|
→ back to step 2, with the new range

Nothing here is triggered manually — there is no "refresh" button, no event listener wiring up a redraw. The dependency graph (query → property → computed/binding → screen) does it all. That reactive model is the single biggest idea to convey.

A15. Worked example — trace it end to end

Setup: baseline picker = July 2026, end picker = September 2026, user clicks Apply.

_applyBaseline() runs:

pick = '2026-07' endPick = '2026-09'

```
MONTH.test('2026-07') → true      MONTH.test('2026-09') → true
'2026-09' < '2026-07' → false    (no error)
```

```
setProperties({
  baselineStart: '2026-07-01',
  baselineEnd:   _monthEndDate('2026-09') → '2026-09-30',
  baselineLabel: _monthYearLabel('2026-07') → 'July 2026',
  baselineEndLabel: _monthYearLabel('2026-09') → 'September 2026'
})
```

The two ES queries fire (once each, in parallel) with start-date=2026-07-01, end-date=2026-09-30, and return:

```
baselineDocsPerMonth = [
  { key: '2026-07', value: 1000 },
  { key: '2026-08', value: 1500 },
  { key: '2026-09', value: 750 }
]
baselineStoragePerMonth = [
  { key: '2026-07', value: 524288000 }, // 500 MB
  { key: '2026-08', value: 786432000 }, // 750 MB
  { key: '2026-09', value: 419430400 } // 400 MB
]
```

_baselineValues(storage, docs) runs:

```
unit = _storageUnit(storage) → max value = 786432000 → { name:'MB',
divisor:1048576 }
storageVals: [524288000, 786432000, 419430400] / 1048576 → [500,
750, 400]
docMap = { '2026-07':1000, '2026-08':1500, '2026-09':750 }
docVals: walk storage's months, look each up → [1000, 1500, 750]
returns: [[500, 750, 400], [1000, 1500, 750]]
```

→ the chart draws three months, each with a storage bar and a document bar.

_baselineSeries(storage) runs:

```
returns ['Storage added in (MB)', 'Number of documents created']
```

_computeBaselineGrowth(storage, docs) runs:

```
baseDocs = docMap['2026-07'] = 1000
baseStorage = storage[0].value = 524288000
```

```
Row 0 (July)      i===0 → docsPct='baseline', storagePct='baseline'
```

```
Row 1 (August)
```

```
  pct(1500, 1000, fmtDocs)    = (500/1000)*100 = +50 → '+50%'
```

```
  pct(786432000, 524288000, fmtStorage) = +50      → '+50%'
```

```
Row 2 (September)
```

```
  pct(750, 1000, fmtDocs)      = (-250/1000)*100 = -25 → '-25%'
```

```
  pct(419430400, 524288000, fmtStorage) = -20      → '-20%'
```

Final _baselineGrowth:

```
[
  { month:'Jul-26', docs:'1,000', storage:'500 MB',
docsPct:'baseline', storagePct:'baseline' },
  { month:'Aug-26', docs:'1,500', storage:'750 MB', docsPct:'+50%',
storagePct:'+50%' },
  { month:'Sep-26', docs:'750', storage:'400 MB', docsPct:'-25%',
storagePct:'-20%' }
]
```

On screen:

Month	Documents	vs Jul	Storage	vs Jul
Jul-26	1,000	baseline	500 MB	baseline

Aug-26	1,500	+50% 	750 MB	+50% 
Sep-26	750	-25% 	400 MB	-20% 

A second trace — the zero-baseline case, since it's the fix your lead will likely ask you to explain: baseline = January 2026 (0 documents), and June 2026 has 116 documents / 452.9 KB.

```
baseDocs = 0    baseStorage = 0
Row (June): pct(116, 0, fmtDocs)
  → base is 0 → current(116) > 0 → '+' + fmtDocs(116) + ' new' →
  '+116 new'
                pct(463770, 0, fmtStorage)
  → '+' + fmtStorage(463770) + ' new' → '+452.9 KB new'
```

→ table shows **+116 new** and **+452.9 KB new**, both green (first character is +), instead of a division-by-zero n/a.

A16. Performance & scale notes specific to the baseline

1. **Two Elasticsearch aggregations per Apply, zero on every keystroke.** The picker inputs are bound to `baselinePick/baselineEndPick`; the queries are bound to the separate `baselineStart/baselineEnd`. Typing in the box costs nothing.
2. **One notification cycle per Apply**, via `setProperties`, so the two queries never fire twice for a single click.
3. **No document bodies are fetched.** Both queries are aggregations (bucketed count / sum) — the response is a handful of `{key, value}` pairs regardless of whether the repository holds a thousand or a billion documents. Elasticsearch aggregations scale with the number of *buckets* (at most however many months are in the picked range), not with corpus size.
4. **All client-side computation after the query is $O(n)$ in months** — a `.map()` here, a `.reduce()` there, over an array that is realistically never more than a few dozen entries. This is negligible compared to the network round-trip.

PART B — THE EMAIL REPORT

B1. The button markup

```
<div class="email-report-row">
  <span class="email-report-label">
    <svg viewBox="0 0 24 24">...</svg>
    Email this report
  </span>
  <button type="button" class="email-btn" on-tap="_emailReport">
    <svg viewBox="0 0 24 24">...</svg>
    Send report
  </button>
  <span class="email-status">[[emailStatus]]</span>
</div>
```

Same on-tap binding pattern as the baseline's Apply button. `[[emailStatus]]` is a one-way binding that displays progress text as the report is built and sent — the user sees "Building PDF..." → "Attaching charts..." → "Sending..." → "Report sent" without any custom progress-bar code; it's just a string property being updated at each stage, and the binding keeps the screen in sync automatically.

B2. The property

```
emailStatus: { type: String, value: '' },
```

One plain string property. Every status message in `_emailReport` is just an assignment to `this.emailStatus`.

B3. `_emailReport()` — walked through in six stages

This is the largest method in the component. It breaks cleanly into six stages; each is explained separately, then the whole picture is drawn together in the flow diagram (B6).

Stage 1 — setup and the headline numbers

```
_emailReport: function() {
  var self = this;
  var email = 'Shreyas.BJoshi@fmr.com'; // recipient
  var FROM = 'noreply@fmr.com';        // must match your
  mail.from

  this.emailStatus = 'Preparing report...';

  var total    = this.totalCount || 0;
  var migrated = this.migratedCount || 0;
  var nonMig   = (total - migrated) > 0 ? (total - migrated) : 0;
  var pct      = total ? Math.round((migrated / total) * 100) : 0;

  var withCommas = this._withCommas.bind(this);
  var fmtStorage = this._formatStorage.bind(this);
```

`var self = this;` — the Part-0 pattern again; this method is full of nested callbacks (`.then()`, `.forEach()`), each of which needs `self` to reach back to the component.

`this._withCommas.bind(this)` — a JavaScript concept worth naming explicitly to your lead: `.bind()`. `_withCommas` is a method that internally might rely on `this` (though this particular one doesn't, others like `_formatStorage` are pure). `.bind(this)` creates a **new function** that, no matter how or where it's later called, always has `this` locked to the current component. This lets the email body use `withCommas(...)` and `fmtStorage(...)` as plain function calls throughout the rest of the method, rather than writing `self._withCommas(...)` every time — cleaner code, same correctness guarantee.

`(total - migrated) > 0 ? (total - migrated) : 0` — a defensive guard so "non-migrated" can never go negative (which would otherwise be possible only from a data inconsistency, but it's cheap insurance).

Stage 2 — building the HTML email body

```
var html = '<div style="font-family:Arial,sans-
serif;color:#20302A;">'
  + '<h2 style="color:#1E5631;margin:0 0 4px;">Nuxeo Usage
Report</h2>'
  + '<p style="color:#5A6560;margin:0 0 16px;">Period: ' +
  (this.startDate || '') + ' to ' + (this.endDate || '') + '</p>'
  + '<table cellpadding="8" style="border-collapse:collapse;font-
size:14px;">'
  + '<tr>...Total documents added...' + withCommas(total) + '</td>'
  + '</tr>'
  + ...
```


This is **string concatenation** building an HTML document, piece by piece, with +=. There is no templating engine here — it's plain JavaScript strings. The reason every element has an inline style="..." attribute (rather than a <style> block or CSS classes) is a **hard requirement of HTML email**: most email clients strip <style> blocks and external stylesheets entirely, so the *only* reliable way to style an email is inline, element by element.

The same pattern repeats for the **Top 5 by documents**, **Top 5 by storage**, and **Growth vs Baseline** sections — each is: check the relevant array has data, if so append a <h3> and a <table>, loop with a plain for (Top 5) or .forEach() (baseline rows) to append one <tr> per entry.

```
var bg = this._baselineGrowth || [];
if (bg.length) {
  var blabel = this.baselineLabel || 'baseline';
  var bshort = this.baselineShort(this.baselineLabel);
  html += '<h3 ...>Growth vs ' + blabel + ' (Baseline)</h3>' + ... ;
  bg.forEach(function(row) {
    html += '<tr><td>' + row.month + '</td>...';
  });
  html += '</table>';
}
```

This is the direct payoff of Part A's _baselineGrowth — the email doesn't recompute anything; it reads the *same* computed array the on-screen table already built, so whatever baseline range and numbers the user is looking at is exactly what gets emailed. The heading text (blabel) and column headers (bshort) pull from the same baselineLabel property too — change the baseline picker, and the email automatically reflects the new choice without any separate email-specific logic needing to change.

Stage 3 — the sendMail inner function

```
function sendMail(tempDoc) {
  self.emailStatus = 'Sending...';
  var params = {
    from: FROM,
    to: email,
    subject: 'Nuxeo Usage Report - ' + (self.startDate || '') + ' to ' + (self.endDate || ''),
    message: html,
    HTML: true,
    rollbackOnError: false
  };
  var body = { input: 'doc:', params: params };
  if (tempDoc && tempDoc.uid) {
    body.input = 'doc:' + tempDoc.uid; // the doc holding the PDF
    params.files = 'file:content'; // attach its blob
  }
  return fetch('/nuxeo/api/v1/automation/Notification.SendMail', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json', 'Accept': 'application/json' },
    credentials: 'include',
    body: JSON.stringify(body)
  }).then(function(resp) {
    if (resp.ok || resp.status === 204) {
      self.emailStatus = 'Report sent';
    } else {
      return resp.text().then(function(t) {
        if (window.console) { console.error('SendMail failed', resp.status, t); }
        self.emailStatus = 'Could not send. Please try again.';
      });
    }
  });
}
```

```
});
}
```

This is a **function declared inside another function** — a **closure**. It captures `self`, `html`, `email`, and `FROM` from the outer `_emailReport` scope, so it can be called later (twice, potentially — with and without a PDF) without needing those values passed in again.

`fetch(...)` is the browser's built-in API for making HTTP requests — this is how the browser talks to the Nuxeo server. It POSTs a JSON body to Nuxeo's **Automation REST endpoint** for the built-in `Notification.SendMail` operation.

Why call a built-in operation from the browser, rather than a custom server-side script? This Nuxeo instance's server-side scripting engine (Nashorn) is broken (a known platform issue on this deployment, unrelated to this component). Built-in Java operations like `Notification.SendMail` still work fine; only *custom scripted* operations fail. So the design deliberately does all the assembly (the HTML, the PDF) in the **browser**, and only calls proven built-in operations on the server.

The input/params shape is Nuxeo's Automation calling

convention: every operation call is `{ input: <a document reference>, params: {...} }`. `'doc:/'` means "the repository root" — `Notification.SendMail` needs *some* document as its context even when just sending plain text, and the root satisfies that with no side effects.

`if (tempDoc && tempDoc.uid) { ... }` — this is where the PDF gets attached, if one was built. `files: 'file:content'` is an **xpath**: it tells `SendMail`, "look at the document given as input, and attach whatever blob lives at its `file:content` field." This only works because Stage 5 below first uploads the PDF *into* a real document (`SendMail` cannot attach a blob that isn't already stored on some document).

`.then(function(res) {...})` — `fetch` returns a **Promise** (explained fully in the glossary, B9). This `.then()` runs once the server responds: `resp.ok` covers the 200-299 status range, 204 is added explicitly because Nuxeo's `SendMail` can return "No Content" on success. Any other status is treated as failure, logs the real error text to the console for debugging, and shows a generic, non-alarming message in the UI.

Stage 4 — generating the PDF (this is where the real engineering is)

```
try { this.activeView = 'overall'; } catch (e) {}

this._ensurePdfLibs().then(function() {
  return new Promise(function(res) { setTimeout(res, 500); });
}).then(function() {
  var root = self.shadowRoot || self;

  function shot(heading) {
    var cards = root.querySelectorAll('nuxeo-card');
    for (var i = 0; i < cards.length; i++) {
      var h = cards[i].getAttribute &&
cards[i].getAttribute('heading');
      if (h !== heading) continue;
      var c = cards[i].querySelector('canvas');
      ...
      if (c && c.width && c.height) {
        var tmp = document.createElement('canvas');
        tmp.width = c.width; tmp.height = c.height;
        var ctx = tmp.getContext('2d');
        ctx.fillStyle = '#ffffff';
        ctx.fillRect(0, 0, tmp.width, tmp.height);
        ctx.drawImage(c, 0, 0);
        return { img: tmp.toDataURL('image/png'), w: c.width, h:

```

```

c.height };
    }
  }
  return null;
}
...

```

`this.activeView = 'overall'` — switches the Growth Overview card to its "Overall" tab before capturing, so the exported chart shows documents + storage combined rather than whatever tab the user happened to be on.

`_ensurePdfLibs()` then a `setTimeout` wrapped in a new `Promise` — loads the PDF-generation library (jsPDF, from a CDN, only once — see B4), then waits 500 milliseconds. That delay exists because switching `activeView` triggers Chart.js to redraw the Overall chart; without a brief pause, capture could run *before* the redraw finishes, grabbing a stale or blank canvas. `new Promise(function(res){ setTimeout(res, 500); })` is the standard idiom for "turn a timed delay into something a Promise chain can wait on."

Why chart images are captured via `canvas.toDataURL()`, and specifically NOT via `html2canvas`: an earlier version of this feature used `html2canvas` to screenshot each chart *card* (to capture the numbers around the chart, not just the bare graph). In production testing, `html2canvas` **froze the browser's main thread** on these Chart.js canvases — sometimes for tens of minutes, with the UI stuck showing "Capturing... 2/3" indefinitely. `canvas.toDataURL()` is a **native browser API** that reads pixel data directly off an existing canvas — it never hangs, because it does no DOM walking or re-rendering; it just copies bytes. This was a deliberate, hard-learned architectural decision, and is very likely something your lead will ask about if they've seen similar screenshot-library issues before.

The trade-off this creates: `toDataURL()` only captures the `<canvas>` element itself. Numbers that live in **HTML around** the canvas (like a donut chart's centre percentage, which is a `<div>` overlaid on top of the canvas, not drawn inside it) are **not** included. That gap is closed deliberately in Stage 5 — the missing numbers are re-drawn onto the PDF as text, next to the chart image, rather than being screenshotted.

Why draw onto a second, temporary canvas first (tmp), instead of calling `toDataURL()` on the chart's own canvas directly? Two reasons in one block: (1) Chart.js canvases are often **transparent**, and PNG preserves transparency — pasted into a PDF, a transparent background can render as black. Drawing white onto tmp first, then the chart on top, flattens it to a guaranteed-white background. (2) it leaves the on-screen chart completely untouched — nothing about what the user sees is modified during capture.

Why PNG, not JPEG: this was tested, not assumed. For flat-colour chart graphics (solid bars, sharp lines, few colours), PNG compresses to roughly 5× **smaller** than JPEG — JPEG is optimised for photographic gradients, and actively performs worse on this kind of image. An earlier attempt "optimised" to JPEG on the assumption it would be smaller, was measured, and reverted when the numbers showed the opposite.

Stage 5 — building the PDF pages and drawing the missing numbers

```

var pdf = new window.jspdf.jsPDF({ orientation: 'p', unit: 'pt',
format: 'a4', compress: true });
...
pages.forEach(function(pg) {
  if (!pg.chart) return;
  pdf.addPage(); added++;
  pdf.text(pg.title, M, 52);

```

```

...
pdf.drawImage(pg.chart.img, 'PNG', imgX, 88, w, h);

if (pg.donutCenter) {
  var cx = imgX + w / 2;
  var cy = 88 + h / 2;
  pdf.text(pg.donutCenter.pct, cx, cy, { align: 'center' });
  pdf.text(pg.donutCenter.label, cx, cy + 14, { align: 'center'
});
}

pg.stats.forEach(function(st) {
  if (ty > ph - 40) return;
  pdf.text(String(st[0]), M + 6, ty);
  pdf.text(String(st[1]), M + 190, ty);
  ty += 19;
});
});

```

jsPDF is a library that builds a PDF document programmatically — every `pdf.text(...)` and `pdf.drawImage(...)` call places content at explicit x/y coordinates on the page, in points (unit: 'pt').

`pages` is an array of plain JavaScript objects (built just above this snippet), one per chart: { title, sub, chart, stats, donutCenter? }. This is the "one page per chart" design — a cover/summary page, then one page per captured chart, skipping any chart that failed to export (if (!pg.chart) return; — a defensive guard so one bad capture doesn't break the whole PDF).

`if (pg.donutCenter) { ... }` — this is the direct fix for the gap noted in Stage 4. The donut's canvas image has an empty hole in the middle (the percentage text was never part of the canvas). This block draws that percentage as **PDF text**, positioned mathematically at the exact centre of the placed image ($\text{imgX} + w/2, 88 + h/2$) — restoring the missing number without needing a screenshot library at all.

`pg.stats.forEach(...)` — every other "number that lives outside the canvas" (Migrated count, Non-migrated count, Total, baseline growth rows) is handled the same way: drawn as plain text beneath the chart image, using the same numbers already computed for the on-screen dashboard and the email body — one source of truth, three destinations (screen, email body, PDF).

```

var pdfBlob = pdf.output('blob');
var sizeKB = Math.round(pdfBlob.size / 1024);
if (sizeKB > 4000) { throw new Error('PDF too large to email (' +
sizeKB + ' KB)'); }

```

`pdf.output('blob')` converts the assembled PDF into a binary **Blob** (browser-native binary data object) ready to upload. The size guard exists because of a real production incident: an earlier `html2canvas`-based PDF came out at **5.4 MB**, and the mail server silently refused to send an attachment that large. Throwing here routes the flow into the `.catch()` at the very end (Stage 6), which falls back to sending the numbers-only email rather than failing with no message at all.

Stage 6 — uploading the PDF and attaching it

```

self.emailStatus = 'Attaching charts...';
return fetch('/nuxeo/api/v1/upload/', { method: 'POST', credentials:
'include' })
.then(function(r) { if (!r.ok) throw new Error('batch create ' +
r.status); return r.json(); })
.then(function(batch) {
  var batchId = batch.batchId;
  return fetch('/nuxeo/api/v1/upload/' + batchId + '/0', {
    method: 'POST', credentials: 'include',

```

```

        headers: { 'Content-Type': 'application/pdf', 'X-Upload-Type':
'normal', 'X-File-Name': 'usage-report.pdf' },
        body: pdfBlob
    }).then(function(u) { if (!u.ok) throw new Error('upload ' +
u.status); return batchId; });
    })
    .then(function(batchId) {
        self.emailStatus = 'Preparing attachment...';
        var createBody = {
            input: 'doc:/',
            params: {
                type: 'File',
                name: 'usage-report-' + Date.now(),
                properties: 'dc:title=Usage Report\\nfile:content=' +
JSON.stringify({ 'upload-batch': batchId, 'upload-fileId': '0' })
            }
        };
        return fetch('/nuxeo/api/v1/automation/Document.Create', { ...
    })
        .then(function(r) { if (!r.ok) throw new
Error('Document.Create ' + r.status); return r.json(); });
    });

```

This is Nuxeo's standard **three-step blob-upload dance**, and it's worth being able to narrate as a sequence:

1. **POST /api/v1/upload/** — creates an empty *upload batch* on the server and returns a batchId. Think of this as "reserving a slot."
2. **POST /api/v1/upload/<batchId>/0** — uploads the actual PDF bytes (body: pdfBlob) into slot 0 of that batch. The headers tell Nuxeo the content type and the filename.
3. **Document.Create** — creates a real File document in the repository, whose file:content field is populated **by reference to the batch** ({ 'upload-batch': batchId, 'upload-fileId': '0' } — not by re-sending the bytes). This is why the earlier Notification.SendMail call in Stage 3 could attach it: SendMail can attach a blob that lives on a *document*, but cannot attach a freshly-uploaded batch blob directly — hence the batch has to become a real document first.

Each step's .then() checks r.ok and throws a descriptive error if not — every failure has a distinguishable message ('batch create 500', 'upload 413', 'Document.Create 403'), which is what appears in the browser console if something goes wrong, making the failure mode diagnosable rather than a silent dead end.

Tying Stages 4-6 and the two possible endings together

```

    }).then(function(tempDoc) {
        return sendMail(tempDoc); // email WITH the PDF attached
    }).catch(function(err) {
        if (window.console) { console.error('PDF step failed, sending
figures only:', err); }
        self.emailStatus = 'Sending...';
        return sendMail(null);
    });

```

The **entire** Stage 4→5→6 sequence is one long .then().then().then()... chain. If it completes successfully, its final result (tempDoc, the created File document) flows into sendMail(tempDoc) — the email goes out **with** the PDF attached.

.catch(function(err) {...}) at the very end catches a failure from **anywhere** in that entire chain — a library failing to load, a chart failing to capture, the PDF coming out oversized, any of the three upload steps failing. No matter which step breaks, the same fallback runs: sendMail(null) — the email still goes out, with the numbers and

tables in the body, just without the PDF attachment. **The user is never left with a silently failed report; the report always sends, in either its best form or a slightly reduced one.**

B4. `_ensurePdfLibs()` — loading the PDF library on demand

```
_ensurePdfLibs: function() {  
  if (window.jspdf) { return Promise.resolve(); }  
  return new Promise(function(res, rej) {  
    var s = document.createElement('script');  
    s.src =  
    'https://cdnjs.cloudflare.com/ajax/libs/jspdf/2.5.1/jspdf.umd.min.js';  
  
    s.onload = res; s.onerror = rej;  
    document.head.appendChild(s);  
  });  
},
```

`window.jspdf` is checked first — if the library is already loaded (from a previous email send in the same session), skip straight to `Promise.resolve()` (an already-completed Promise, so the chain continues immediately with no delay). Otherwise, a `<script>` tag is created dynamically and appended to the page's `<head>`, which makes the browser fetch and execute the library from the CDN. `s.onload = res` resolves the Promise once the script has finished loading; `s.onerror = rej` rejects it (routing into the `.catch()` in `_emailReport`) if the CDN is unreachable.

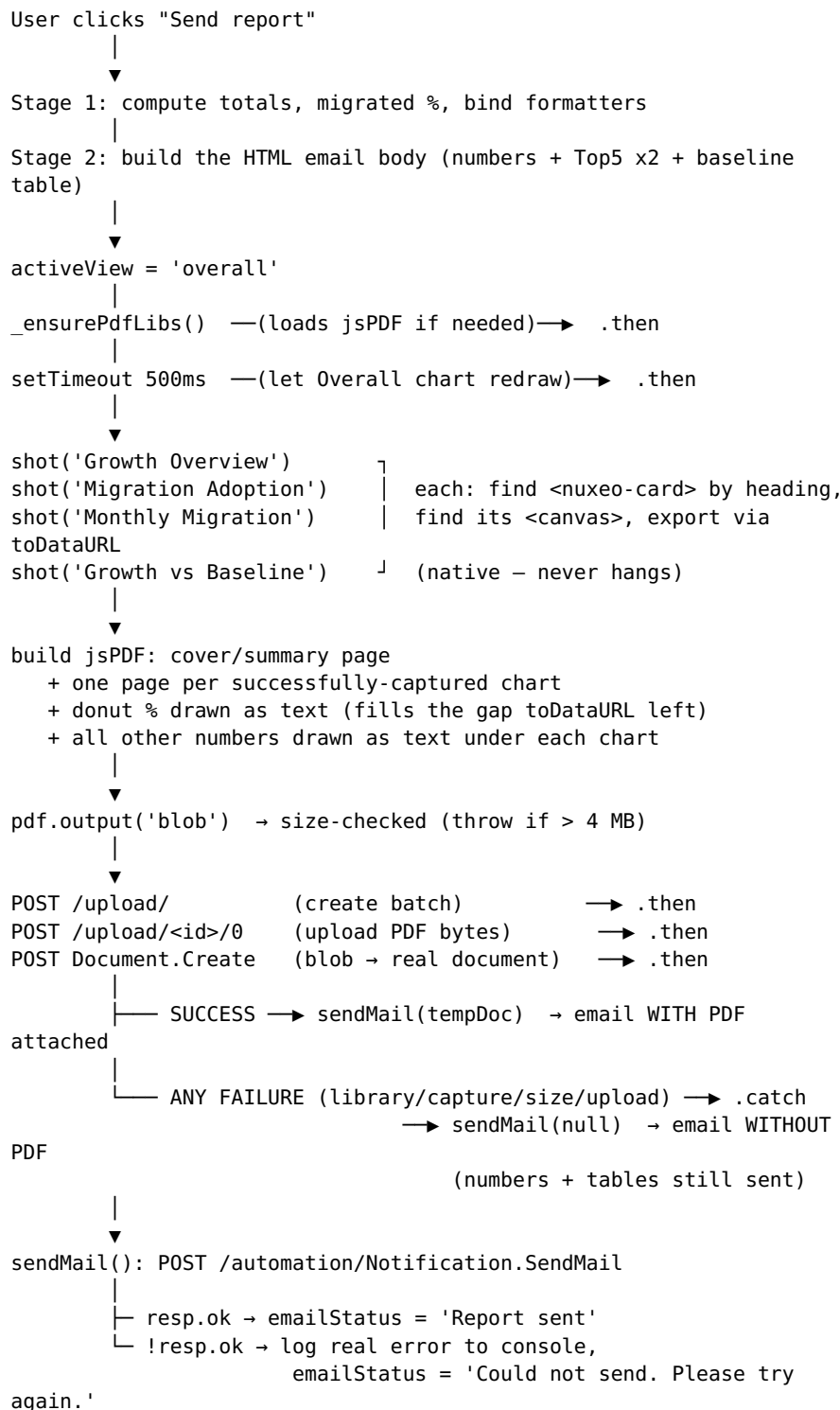
Note: this file previously also loaded `html2canvas` here. It was removed entirely once the native `canvas.toDataURL()` approach replaced it — one fewer external dependency, one fewer thing that can hang.

B5. The email CSS

```
.email-report-row {  
  display: flex;  
  align-items: center;  
  gap: 12px;  
  padding: 14px 18px;  
  background: #f3f9f3;  
  border-left: 4px solid #2e7d32;  
  border-radius: 8px;  
}  
.email-btn {  
  background: #2e7d32;  
  color: #ffffff;  
  ...  
}  
.email-btn:hover { background: #256628; }  
.email-btn:active { transform: translateY(1px); }
```

Standard component styling — a highlighted row (light green background, green left border to draw the eye), a solid button with hover/active states for tactile feedback. Nothing structurally unusual here; it's worth mentioning only that this CSS is **entirely separate** from the *inline* styles inside the email HTML string in Stage 2 — the dashboard's own CSS never reaches the email, because the email is a standalone document sent to a different context (an inbox), so it has to carry every style itself.

B6. The complete program flow (the Promise chain, visually)



B7. Worked example — tracing the email for a real send

State: period = July-September 2026, totalCount = 15416, migratedCount = 5095, baseline = July 2026 as computed in A15.

Stage 1:

total=15416 migrated=5095

```
nonMig = (15416-5095)>0 ? 10321 : 0 → 10321
pct = Math.round((5095/15416)*100) → 33
```

Stage 2 (excerpt of the numbers table row built):

```
'<tr><td><b>Migrated</b></td><td>' + withCommas(5095) + ' (' + 33 +
'%)</td></tr>'
→ '<tr><td><b>Migrated</b></td><td>5,095 (33%)</td></tr>'
```

Baseline section (reusing A15's _baselineGrowth):

```
blabel = 'July 2026' bshort = 'Jul'
row 2 (Aug-26): '<tr><td>Aug-26</td><td align="right">1,500</td>
<td align="right">+50%</td><td align="right">750
MB</td>
<td align="right">+50%</td></tr>'
```

— identical numbers to what's on screen, because it's the same array.

Stage 4: the Migration Adoption donut is captured. Its canvas shows the ring and legend, but the centre is blank pixels (the "33% MIGRATED" text was never part of the canvas). `pg.donutCenter = { pct: '33%', label: 'MIGRATED' }` is carried alongside the captured image.

Stage 5: when this page is placed in the PDF, after `pdf.drawImage(...)` draws the ring, the donut-centre block runs:

```
cx = imgX + w/2 cy = 88 + h/2
pdf.text('33%', cx, cy, {align:'center'})
pdf.text('MIGRATED', cx, cy+14, {align:'center'})
```

→ the PDF page shows a complete donut with "33% / MIGRATED" sitting exactly in its centre — reconstructed as text, not screenshotted.

Stage 6: the finished PDF (four chart pages + summary, all native PNGs, each roughly 15 KB) is measured — say **310 KB total**, well under the 4 MB guard — uploaded in the three-step dance, wrapped in a File document.

Final `sendMail(tempDoc)` call:

```
{
  input: 'doc:<uid-of-the-new-File-document>',
  params: {
    from: 'noreply@fmr.com',
    to: 'Shreyas.BJoshi@fmr.com',
    subject: 'Nuxeo Usage Report - 2026-07-01 to 2026-09-30',
    message: '<the full HTML string from Stage 2>',
    HTML: true,
    rollbackOnError: false,
    files: 'file:content'
  }
}
```

POSTed to `Notification.SendMail`. Server responds 200 → `emailStatus = 'Report sent'`.

B8. Performance considerations for the email/PDF path

1. **No blocking of the page or the ES queries.** Everything in `_emailReport` runs in a Promise chain — genuinely asynchronous. The dashboard's own charts and data are entirely unaffected while the email is being built; nothing here adds Elasticsearch load (the numbers are pulled from properties already in memory).

2. **`canvas.toDataURL()` is $O(\text{pixels})$, not $O(\text{DOM})$.** It never hangs, unlike a screenshot library that has to walk and re-render arbitrary DOM trees. This was the direct fix for a production incident where the UI froze for 30+ minutes.
 3. **PNG over JPEG for these specific images** — measured, not assumed, and roughly $5\times$ smaller for flat-colour charts, which is what keeps the whole PDF in the tens-to-low-hundreds of KB range instead of megabytes.
 4. **The 4 MB size guard** exists because of a real prior failure (a 5.4 MB attachment the mail server silently rejected) — a concrete, previously observed failure mode, not speculative defensive coding.
 5. **The `.catch()` guarantees the report is never lost** — worst case, the PDF is skipped and a plain numbers email still goes out. From a reliability standpoint, this converts "the feature might fail visibly" into "the feature always degrades gracefully."
-

B9. Glossary — the JavaScript / Polymer concepts a lead may probe

Promise — an object representing a value that isn't ready yet, but will be (or will fail) at some point in the future. `fetch()`, timers wrapped in `new Promise`, and script-loading all return Promises here.

`.then(fn)` — schedules `fn` to run once the Promise it's attached to succeeds; `.then()` itself returns a **new** Promise, which is what allows chaining (`.then().then().then()`) — each step waits for the previous one.

`.catch(fn)` — schedules `fn` to run if **anything** earlier in the chain failed (a network error, a thrown exception, an explicit throw). One `.catch()` at the end of a long chain catches failures from every step before it — this is why Stage 6's three uploads, Stage 5's size guard, and Stage 4's chart capture can all funnel into the single fallback in `_emailReport`.

`fetch(url, options)` — the browser's native HTTP client. Returns a Promise that resolves with a Response object; you then call `.json()` or `.text()` on that response (which are *themselves* asynchronous, hence the extra `.then()` after almost every `fetch`).

Closure — a function that "remembers" variables from the scope it was defined in, even after that outer function has returned. `sendMail` is a closure over `self`, `html`, `email`, `FROM`.

`.bind(this)` — produces a new function permanently locked to a specific `this`, regardless of how it's later invoked. Used here so `_withCommas` and `_formatStorage` can be called as bare functions (`withCommas(...)`) inside the rest of the method.

`self = this` — the manual alternative to `.bind()`, used inside for loops and `.forEach().map()` callbacks throughout this component, because every plain function `(){}` passed as a callback gets its own `this`.

Blob — a browser object representing raw binary data (here, the PDF bytes) — the thing that actually gets uploaded and attached.

`document.createElement('script') + .appendChild` — the standard vanilla-JS technique for loading an external library at runtime, used by `_ensurePdfLibs` instead of a static `<script>` tag, so the library is only downloaded if and when the user actually sends a report.

Polymer on-tap — click/tap event binding, wired declaratively in the template rather than with `addEventListener` in code.

Polymer [[]] vs { { } } — one-way display binding vs two-way writable binding (see Part 0).

Polymer computed property — a property whose value Polymer derives automatically from other properties, re-running only when a listed dependency changes.

Polymer setProperties — batches multiple property assignments into one notification cycle, preventing double-firing of anything bound to more than one of them.

PART C — Anticipated questions and how to answer them

"Why is the baseline separate from the page's date range?"

Because leadership needs to compare growth against a *fixed reference point* that shouldn't move just because someone changes what period they're browsing. The baseline queries are bound to their own `baselineStart/` `baselineEnd` properties, only updated by the baseline's own Apply button.

"What happens if the ES query hasn't returned yet when the page first renders?" Every helper that touches query data starts with a guard clause (`if (!storage || !storage.length) return ...;`), so nothing throws — the chart and table simply render empty until data arrives, then update automatically via the reactive bindings.

"How do you know the document count and storage bars line up for the same month?" They come from two independent queries that aren't guaranteed to return identical month buckets. `_baselineValues` builds a lookup keyed by month and aligns the second array to the first *by key*, never by array position — explained in full in A9.

"Why doesn't a zero-document baseline month show 'n/a' anymore?" Because percentage change is mathematically undefined when the baseline is zero (division by zero), but the growth is real and worth reporting — so the code shows the absolute increase instead (+116 new), which is always computable and honest.

"Why not screenshot the whole dashboard for the PDF?" That was tried first (`html2canvas`) and it froze the browser's main thread on the Chart.js canvases in production — sometimes for 30+ minutes. Switched to native `canvas.toDataURL()`, which never hangs, at the cost of not capturing HTML that sits outside the canvas (fixed by redrawing those numbers as PDF text).

"What happens if the PDF fails to build?" The entire PDF pipeline is one Promise chain with a single `.catch()` at the end. Any failure — library load, chart capture, oversized PDF, any upload step — falls through to sending the plain-numbers email instead. The report is never silently lost.

"Does any of this affect the dashboard's Elasticsearch query load?" No — the email reuses numbers already sitting in component properties from the dashboard's normal queries; it issues no additional ES queries. Its only extra network calls are the PDF library CDN fetch (once per session) and the Nuxeo REST calls for upload/create/send.

"Why JSON in `note:note` / string concatenation for the email instead of a proper template engine?" No server-side scripting is available (Nashorn is broken on this instance), so the email HTML has to be assembled in the browser using plain JavaScript string building

— there's no templating library loaded for this purpose, and pulling one in for a single email body wasn't judged worth the extra dependency.

Appendix — Quick reference

Concept	Syntax	Meaning
One-way binding	<code>[[foo]]</code>	display a value, parent → child
Two-way binding	<code>{{foo}}</code>	child can write back (query results, form inputs)
Computed binding	<code>[_fn(a, b)]</code>	call a method, use its return value, in the template
Computed property	<code>computed: '_fn(a, b)'</code>	auto-recalculate in JS when a/b change
Batched update	<code>this.setProperties({...})</code>	one notification cycle for multiple properties
Conditional render	<code><template is="dom-if" if="..."></code>	element only exists in DOM when true
Loop render	<code><template is="dom-repeat" items="..."></code>	one copy per array element, <code>item = current</code>
Attribute binding	<code>class\$="[[...]]"</code>	required for <code>class</code> , <code>style</code> , <code>href</code> , etc.
Array/Object default	<code>value: function() { return []; }</code>	must be a function, never a literal
Promise	<code>fetch(...).then(...).catch(...)</code>	async value, chainable, one error handler for the whole chain
Closure	function defined inside another	remembers the outer scope's variables
<code>.bind(this)</code>	<code>fn.bind(this)</code>	locks a function's <code>this</code> permanently
<code>self = this</code>	<code>var self = this;</code>	manual fix for <code>this</code> inside <code>function(){} callbacks</code>